

Enterprise Workflow Architecture Patterns

Initial Version (v1.0)

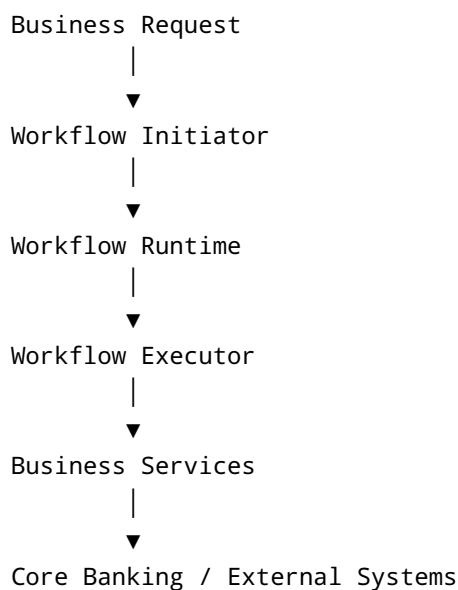
1. Objective

This document introduces common enterprise workflow architecture patterns and shows how Temporal can be used as the workflow orchestration platform.

The goal is to compare different architectural approaches and identify the most suitable pattern for enterprise banking platforms.

2. Common Logical Architecture

Every workflow-based system consists of the same logical responsibilities, regardless of the workflow technology used.



Responsibilities

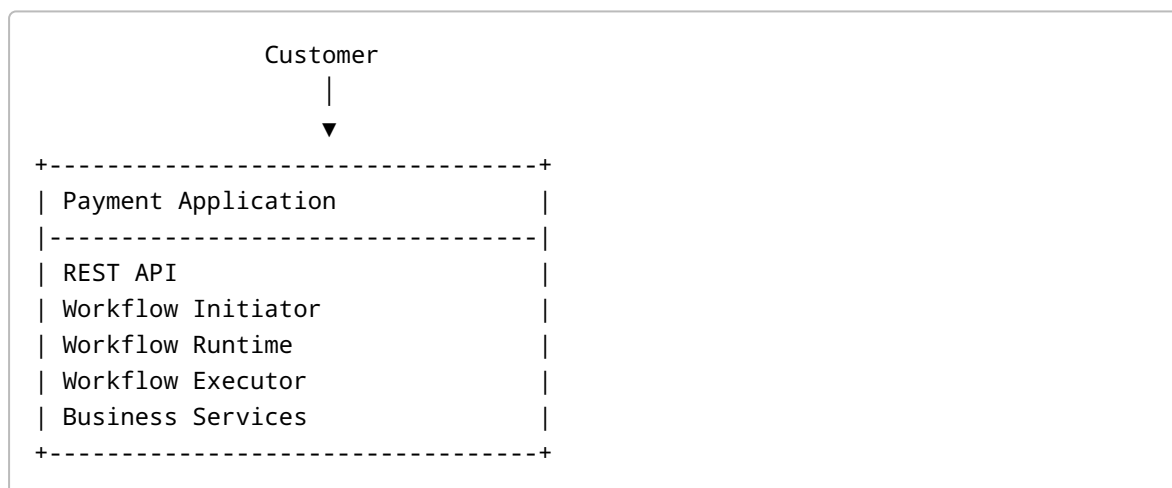
Component	Responsibility
Business Request	Receives a business request from API, UI or AI Agent
Workflow Initiator	Decides when a workflow should start
Workflow Runtime	Coordinates workflow execution, retries, timers and recovery

Component	Responsibility
Workflow Executor	Executes workflow steps
Business Services	Implements business capabilities
Core Banking	System of record

3. Architecture Patterns

Pattern 1 – Embedded Workflow

Architecture



Summary

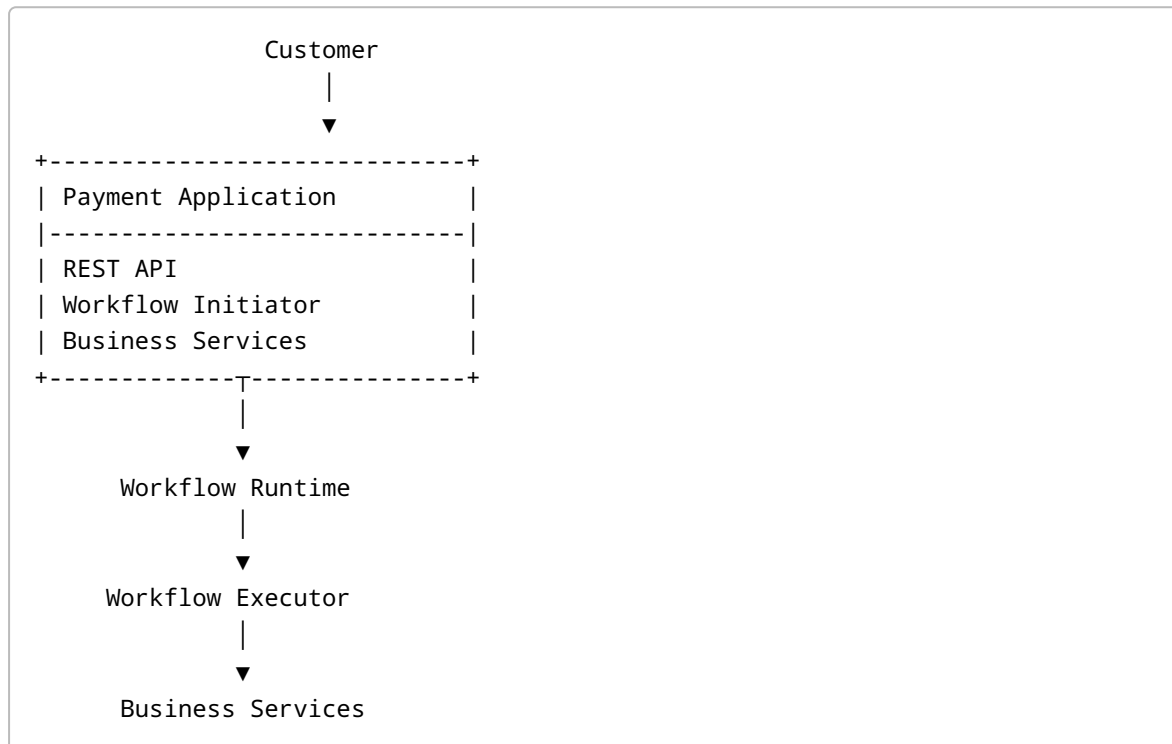
- Everything runs inside one application.
- No external workflow runtime.
- Simplest deployment model.

Best Fit

- Small applications
- Simple workflows
- Internal applications
- Proof of Concepts

Pattern 2 – Central Workflow Runtime

Architecture



Summary

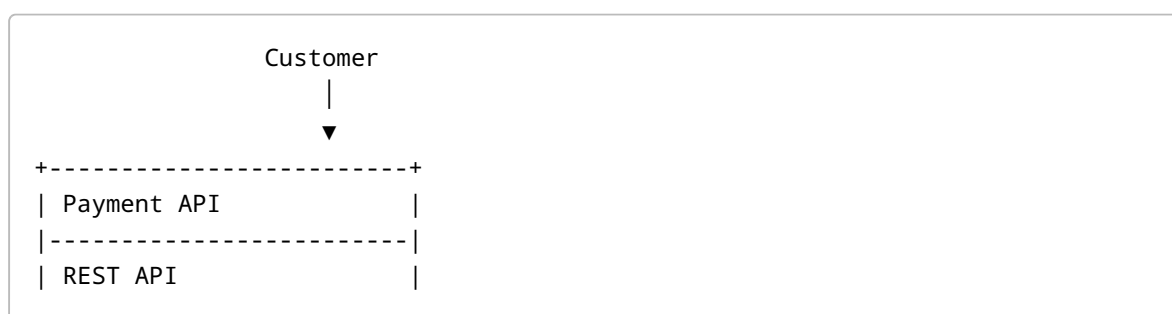
- Business application starts the workflow.
- Workflow coordination is delegated to a central runtime.
- Workflow execution may remain inside the same application.

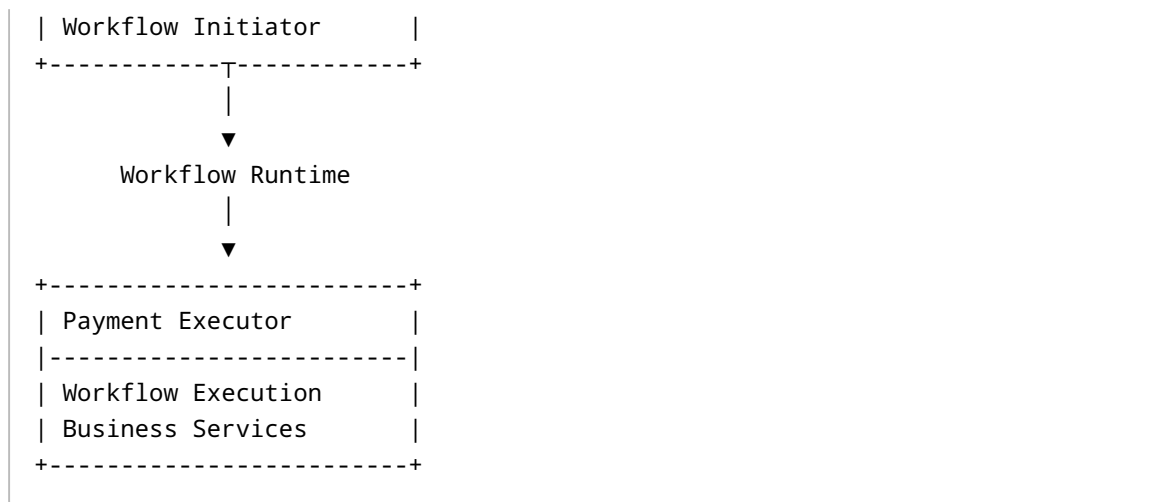
Best Fit

- Growing applications
- Reliable long-running workflows
- Moderate enterprise systems

Pattern 3 – Dedicated Workflow Executors

Architecture





Summary

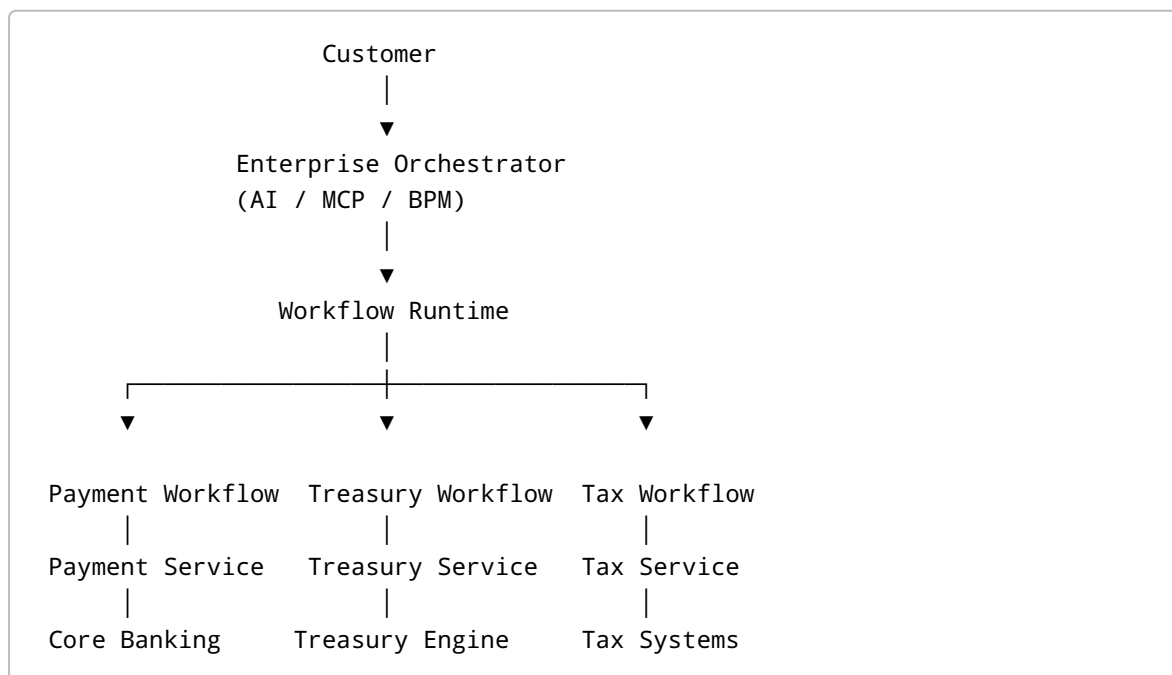
- API initiates workflows.
- Dedicated executors perform workflow execution.
- API and execution scale independently.

Best Fit

- High transaction volume
- Large enterprise applications
- Heavy workflow processing

Pattern 4 – Enterprise Process Orchestration

Architecture



Summary

- Enterprise layer orchestrates cross-domain business processes.
- Each business domain owns its own workflow.
- One workflow runtime coordinates all workflows.

Best Fit

- Enterprise Banking
- AI Agent platforms
- Multi-domain orchestration
- Large organizations

4. Pattern Comparison

Capability	Pattern 1	Pattern 2	Pattern 3	Pattern 4
Workflow Initiation	Application	Application	API	Enterprise Orchestrator
Workflow Coordination	Application	Workflow Runtime	Workflow Runtime	Workflow Runtime
Workflow Execution	Application	Application	Dedicated Executor	Domain Workflow / Domain Services
Independent Scaling	No	No	Yes	Yes
Cross-Domain Orchestration	No	Limited	Limited	Yes
Operational Complexity	Low	Medium	Medium	High
Enterprise Readiness	Low	Medium	High	Very High

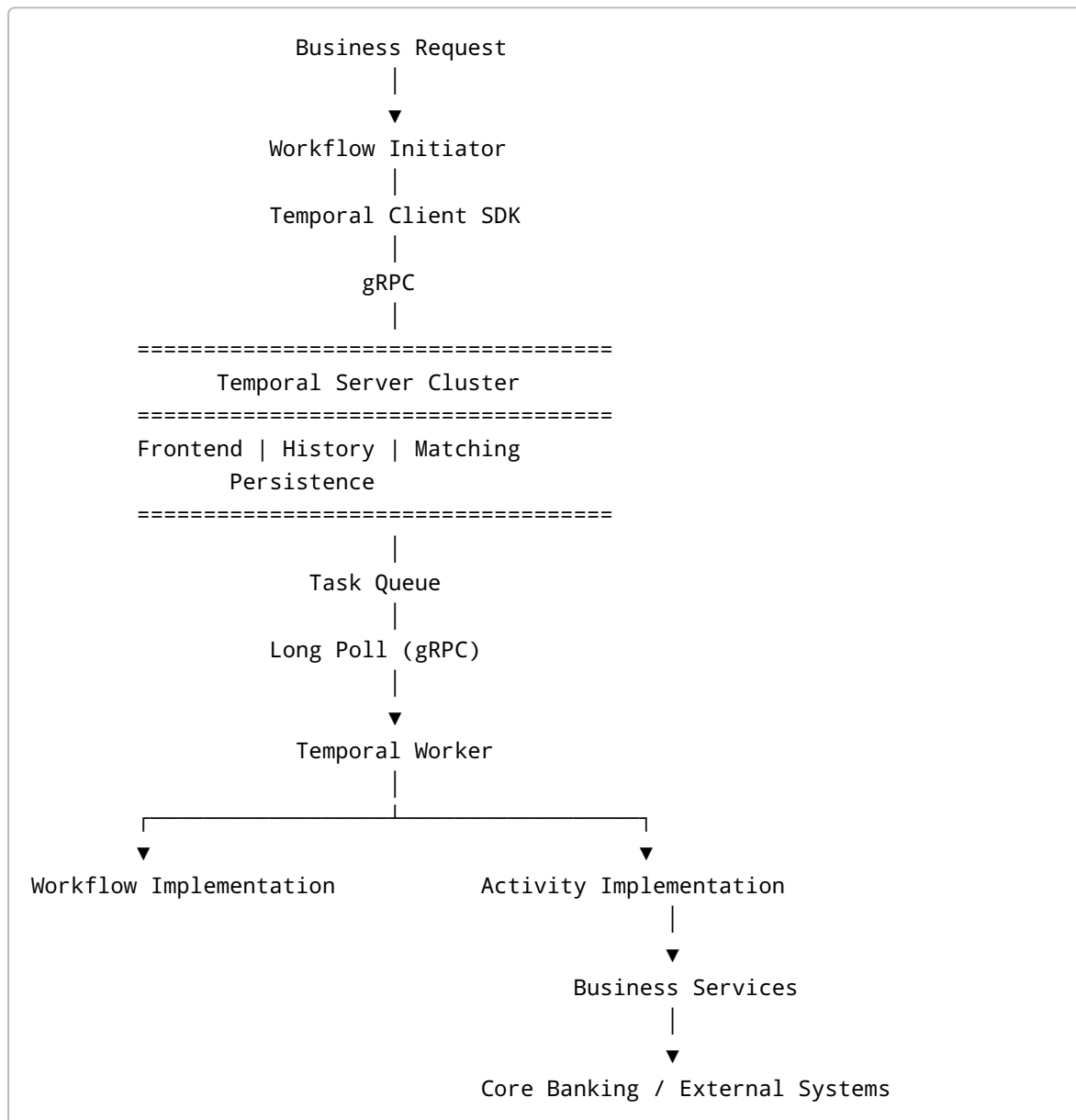
5. Temporal Mapping

The following table shows how the logical architecture maps to Temporal.

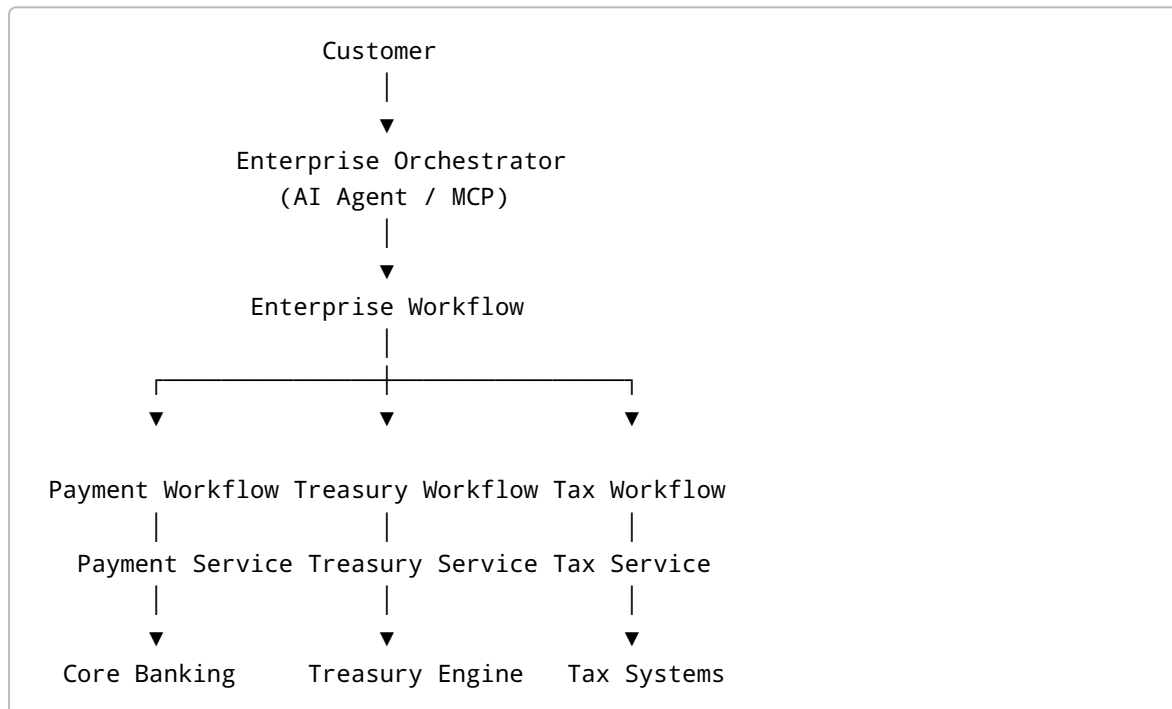
Logical Component	Temporal Component
Workflow Initiator	Temporal Client SDK
Workflow Runtime	Temporal Server
Workflow Executor	Temporal Worker
Workflow Definition	Workflow Implementation

Logical Component	Temporal Component
Business Step	Activity
Business Logic	Activity Implementation

6. Temporal Reference Architecture



7. Recommended Enterprise Banking Architecture



Guiding Principles

- Enterprise Workflow coordinates business processes across multiple domains.
- Each domain owns its own workflow.
- Domain workflows invoke business services.
- Business services own business rules.
- Core systems remain the system of record.

8. Conclusion

There is no single architecture that fits every system.

- **Pattern 1** is ideal for simple applications.
- **Pattern 2** introduces a centralized workflow runtime for reliable execution.
- **Pattern 3** separates workflow execution for better scalability.
- **Pattern 4** provides enterprise-wide orchestration across multiple business domains and is the recommended target architecture for enterprise banking platforms.